Choosing Versus Accepting: Why Better Coding Assistants Can Leave More Technical Debt

Ook Lee, *Hanyang University, Seoul, 04763, South Korea*

Abstract*—Large language model (LLM) coding assistants change the order of a developer's decision. Without an assistant, the developer decides how to build a task before any code exists; with one, a working artifact arrives first and the decision becomes whether to accept it. Drawing on a formal model of this reordering, we show that rising technical debt alone cannot reveal whether developers decide differently, because a rational response to faster work produces the same aggregate pattern. Three statistics can: the share of debt that developers admit to, the effect of holding a working artifact on refactoring, and the rate at which planned refactoring lapses. The model also implies that more reliable models can increase debt, that codebases can tip into debt traps, and that a temporary inspection mandate can have lasting effects. We report commit data consistent with these predictions and draw lessons for teams.

Actionable insights for practitioners

- *Do not judge an assistant by aggregate debt*—Track the share of debt that is self-admitted, whether planned refactoring is carried out, and how much inspection a model upgrade displaces.
- *Decide before generating*—Asking developers to commit to an expedient or a careful approach before the assistant produces code removes the order effect the model identifies.
- *Use inspection mandates to change beliefs*—A mandate that lasts long enough to correct developers' sense of the defect rate can have effects that outlast it, while a shorter one has none.

The standard economic account of technical debt treats each piece of work as a trade-off. A shortcut saves time today and costs rework later, and a team takes it when the time saved is worth more than the rework it will require.[1] On this account, debt should fall when quality becomes cheaper, and LLM coding assistants have made quality much cheaper. The first evidence points the other way. In open-source projects that adopted an agentic assistant, velocity rose only briefly while static-analysis warnings and code complexity rose persistently,[2] and an audit of code generated by LLMs and coding agents found that more capable models produce more bloated and tightly coupled code.[3]

XXXX-XXXX © 2021
Digital Object Identifier 10.1109/XXX.0000.000000

This article argues that assistants change the structure of the quality decision more than its price. Without an assistant, a developer chooses an approach before either implementation exists. With one, a working implementation arrives first, often an expedient one that looks finished, and the decision becomes whether to accept it or to spend further effort improving it. We call this the shift from choosing to accepting. We summarize a formal model of the shift, explain what it implies for how teams should evaluate and oversee assistants, and report repository data that bear on its predictions.

RELATION TO PRIOR WORK

A growing literature documents what happens to generated code. Liu and colleagues traced about 303,000 AI-authored commits across 6,299 repositories and found that 22.7% of the issues those commits introduced were still present in the latest revision.[4] A multivocal review concludes that LLM assistance amplifies code, design, and documentation debt.[5] Among 6,540 code comments that mention LLMs, only about 1.2% also admit technical debt.[6] The role of judgment has been examined as well. Interviews with architects have linked anchoring and optimism to architectural debt,[7] and an observational study of developers working with LLMs classified 48.8% of their actions as biased.[8]

These studies establish that generated code carries persistent issues and that developers rarely record them. They do not explain why, and they cannot say whether the pattern is simply a rational response to cheaper code. Our contribution is to separate the two explanations and to identify what would distinguish them in data.

FROM CHOOSING TO ACCEPTING

The model compares two regimes that differ only in when the developer decides. In the choice regime, the developer commits to an expedient or a careful approach before doing the work. Both the extra effort and the rework it avoids then lie in the future, so the decision depends only on their relative size. In the acceptance regime, an artifact arrives first. The developer may notice that it is defective, may spend time inspecting it, and, if a defect is found, may refactor it, while the rework from an unrepaired defect falls later.

Three well-documented departures from rational choice, which we call wedges, enter the model. Present bias gives immediate costs more weight than future ones.[9] Imperfect perception leads developers to underestimate the defect rate of output that looks finished, as automation bias would predict.[10] Loss aversion makes changing an artifact already in hand feel more costly than building it differently would have been.[11] A fourth wedge, introduced later, is a share of artifacts that are accepted by default without deliberation.

The central observation is that these wedges have nothing to act on in the choice regime. When the decision precedes both the effort and the artifact, present bias scales both sides of the comparison equally, no artifact exists to be misjudged, and nothing is yet in hand to be lost. The same wedges each move the decision once an artifact exists. The order of events, rather than the presence of bias, therefore determines whether the biases matter.

WHY RISING DEBT PROVES LITTLE

Rising debt might seem to confirm the argument, but a rational developer can also accumulate more debt when code becomes cheaper. When a team can build faster, the value of shipping sooner rises, and expedient implementations become more attractive. We call this rational acceleration. The model decomposes the change in debt after adoption into a price effect, which lowers debt; an acceleration effect, which raises it; and a behavioral term, which also raises it. In one

illustration, the price effect lowers the probability that a task ends with debt by 0.24 and the behavioral term raises it by 0.29, so debt rises from 0.500 to 0.555 per task even though quality has become cheaper.

Crucially, for any behavioral configuration there is a faster pace of work at which a fully rational developer produces exactly the same aggregate debt. Aggregate data, including the difference-in-differences estimates that dominate current evidence, therefore cannot tell the two accounts apart. Three statistics can, because each takes a fixed value for a rational developer and shifts in a known direction once a wedge applies (see Table 1):

- *Self-admitted share*—the share of debt in the code that developers have marked, for example with TODO or FIXME comments.[12] It falls below its rational level when perception is imperfect.
- *Possession effect*—how much more often refactoring is deferred when a working artifact is in hand than when the same choice is made in advance. It is zero for a rational developer and positive under present bias or loss aversion.
- *Lapse rate*—the share of planned refactoring that is not carried out. It is positive only under present bias, which separates present bias from loss aversion.

TABLE 1. Predictions that separate rational acceleration from the behavioral account.

Prediction	Rational acceleration	Behavioral account
Aggregate debt after adoption	Rises	Rises
Self-admitted share of debt	Unchanged, no difference by authorship	Lower for generation-first changes
Possession effect on refactoring	Zero	Positive
Lapse of planned refactoring	None	Positive
Statistics when the decision precedes generation	Unchanged	Return to rational values
Debt as model reliability improves	Rises over a narrow interval	Rises over a wider, shifted interval
Effect of a temporary inspection mandate after it ends	Ends with the mandate	Can persist

In the illustration above, rational acceleration reproduces the behavioral level of debt when the value of time rises by a factor of about 2.6, yet its self-admitted share stays at its baseline of 0.50 and its possession effect and lapse rate stay at zero. The behavioral configuration yields a self-admitted share of 0.33, a possession effect of 0.31, and a lapse rate of 0.21. A rational developer may also revise a decision because the artifact reveals something about the work ahead. The model bounds that informational channel by the dispersion of the signal, so a possession effect larger than the bound cannot be explained by information alone.

THREE RESULTS THAT MATTER FOR PRACTICE

Better Models Can Leave More Debt

When defects are frequent, developers inspect nearly every artifact and debt stays low. As the model improves, perceived risk falls and inspection stops before the defect rate has fallen far enough to compensate. Over an interval, the share of defects that reach the codebase then rises faster than the defect rate falls. Even a rational developer relaxes inspection in this way, but the wedges widen the interval and shift it toward the defect rates of today's models. In the illustration in Figure 1, debt per task rises by 56% as the defect rate falls from 0.46 to 0.29. The

same upgrade can therefore reduce debt in one organization and raise it in another.

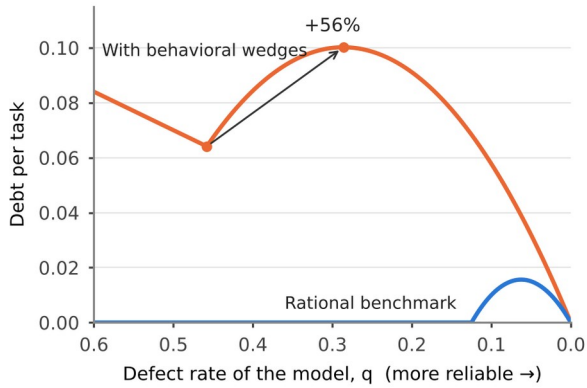


FIGURE 1. The reliability paradox. Debt per task as the defect rate of the model falls, in the rational benchmark and with the behavioral wedges (present bias 0.5, perception 0.6, loss aversion 1.2, default acceptance 0.2). With the wedges, debt rises by 56% as the defect rate falls from 0.46 to 0.29.

Codebases Can Tip Into Debt Traps

Debt is a stock, and a larger stock makes each refactoring more expensive, which is the interest in the metaphor. When present bias applies, a codebase can have two stable states, one with little debt and one with much more. In one parameterization the high-debt state carries ten times the debt of the low one. Because present bias does not act in the choice regime, adoption itself can tip a codebase into the high-debt state even though the assistant also makes refactoring cheaper. Once there, the codebase does not return when conditions recover slightly. Small nudges that would have prevented the trap cannot reverse it, and escape requires a large change in the decision environment or a one-time refactoring program.

Temporary Mandates Can Have Lasting Effects

Developers can learn the true defect rate, but mainly through the inspection that imperfect perception suppresses. If developers believe generated code is reliable enough not to inspect, they observe no defects and never revise the belief. A mandate that requires inspection for a limited period breaks this loop if it lasts long enough for the belief to cross the threshold at which developers start inspecting on their own. In the model's illustration, a single period of mandatory inspection suffices, and the belief then converges to the true rate. Under rational acceleration, by contrast, the effect of a mandate should end when it is lifted, which gives a direct test.

WHAT REPOSITORY DATA SHOW

Archival data cannot test the mechanisms, since aggregate outcomes do not separate the accounts and the wedges are not observed, but they can be consistent or inconsistent with the predictions. We examined six actively maintained open-source projects with many contributors: Prefect, Cal.com, n8n, Grafana, Home Assistant, and Sentry. A commit was treated as AI-assisted when its message carried a co-author trailer or footer naming a coding assistant such as Claude, Codex, Cursor, or Copilot. We analyzed nonmerge commits from April 2025 to October 2026, excluding bot accounts, nonsource files, generated code, and commits that added more than 5000 lines.

To hold skill and style constant, we compared each developer with themselves. For every developer with at least five AI-labeled commits, we compared those commits with the same developer's unlabeled commits made after the first labeled one, retaining developers with at least five of each. This left 251 developers in five projects and 32,620 commits, of which 10,641 carried an AI label (see Table 2).

TABLE 2. Within-developer comparison of AI-labeled and other commits (251 developers, five projects).

Indicator	AI-labeled commits	Other commits	Developers for whom AI-labeled commits score higher	*p*
Self-admitted debt markers per 1000 added lines, all commits pooled	0.16	0.20	22% (29% lower, 49% tied)	.16[b]
Additions as share of changed lines, median across developers	82.5%	78.8%	61%	.002[a]
Added lines duplicated within the same commit, median across developers	32.3%	33.0%	49%	.93[a]
Changed lines per commit, median across developers	152	85	72%	<.001[a]

[a]Wilcoxon signed-rank test across developers.

[b]Sign test excluding ties.

AI-labeled commits carried about 20% fewer self-admitted debt markers per added line, which matches the direction the self-admitted share predicts, although the within-developer difference was not statistically reliable and fewer markers could also reflect less debt. AI-labeled commits were also more weighted toward additions for 61% of developers, which is consistent with retention of generated code. That difference narrowed and lost statistical reliability among commits adding 10 to 300 lines (57% of 170 developers, p = .23), because AI-labeled commits were considerably larger. Duplication within commits did not differ. Labels capture only disclosed assistance, which biases all comparisons toward zero.

A separate analysis of 431,043 agent-authored and 2,525,598 human pull requests on GitHub, drawing on event logs and the AIDev dataset,[13] points the same way. Within repositories, agent pull requests had a ratio of additions to deletions about 1.8 times higher, carried 0.62 self-admitted markers per 1000 added lines against 1.38 for human pull requests, and in team repositories, excluding Copilot, were 5.9 percentage points less likely to be reviewed. These patterns are consistent with the theory but cannot identify its mechanisms.

IMPLICATIONS FOR TEAMS

Remedies such as debt registers and repayment sprints assume that debt is visible and that it will be repaid when the numbers justify it. Where the wedges bind, less debt is recorded and planned repayment lapses, so those remedies will underperform. The model suggests four alternatives:

- *Decide before generating*—Ask developers to commit to an approach before the assistant produces code. The three statistics then return to their rational values.
- *Evaluate upgrades by the inspection they displace*—Measure review effort and the self-admitted share alongside defect rates whenever a model is upgraded.
- *Use mandates to change beliefs*—Set the length of an inspection mandate by whether it corrects developers' sense of the defect rate, not by how long control is needed.
- *Prevent traps rather than reverse them*—Watch codebases that enter adoption with high debt, since they are the most likely to tip, and budget a one-time refactoring program for those that do.

LIMITATIONS

The model is deliberately minimal, and its results are statements about mechanisms rather than calibrated magnitudes. Tasks are independent, inspection detects defects perfectly, and the debt-trap result relies on a particular form of interest on debt. The present-bias results assume that the quality decision without an assistant is made before effort is

committed. Learning concerns the defect rate and not the fluency of the output, which may resist statistical evidence.

The theory is untested. The repository data are observational, drawn from public projects, and silent on the industrial settings in which most commercial software is built. Agent labels are incomplete, and static analysis only approximates the debt developers experience. The decisive test is an experiment that observes the same decision in a choice, a plan, and an acceptance condition with information held fixed, ideally with professional developers in industry.

CONCLUSION

LLM coding assistants have made code cheap to produce, and the first evidence suggests that they have made systems more expensive to maintain. We have argued that the cause lies in how assistance restructures the decision, since a working artifact now arrives before the quality question is posed. A rational response to faster work can produce the same rise in debt, so the behavioral account must be judged by what it alone predicts: debt that goes unrecorded and repayment that lapses despite plans. Two further warnings follow. Better models do not guarantee less debt, and a codebase that has tipped into a high-debt state will not be rescued by the nudges that would have prevented it.

ACKNOWLEDGMENTS

The author used an AI assistant (Claude, Anthropic) for language editing, for adapting the manuscript to this magazine, for drawing Figure 1 from the model, and for writing the commit-mining scripts behind Table 2. The author provided the model, its results, and the pull request analysis, reviewed all AI-assisted content, and takes full responsibility for the article.

REFERENCES

1. W. Cunningham, "The WyCash portfolio management system," in *Addendum Proc. OOPSLA*, 1992, pp. 29–30.
2. H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu, "Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects," in *Proc. 23rd Int. Conf. Mining Softw. Repositories*, 2026, arXiv:2511.04427.
3. Y. Zhu, N. Tsantalis, and P. C. Rigby, "AI-generated smells: An analysis of code and architecture in LLM and agent-driven development," 2026, arXiv:2605.02741.
4. Y. Liu, R. Widyasari, Y. Zhao, I. C. Irsan, J. Chen, and D. Lo, "Debt behind the AI boom: A large-scale empirical study of AI-generated code in the wild," 2026, arXiv:2603.28592.
5. R. Ehsani, S. Rawal, Y. Cai, and P. Chatterjee, "Faster code, deeper debt? A multivocal literature review on technical debt and its early signs in LLM-assisted software development," *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3820165.
6. A. A. Mujahid and M. M. Imran, "'TODO: Fix the mess Gemini created': Towards understanding GenAI-induced self-admitted technical debt," in *Proc. 9th Int. Conf. Tech. Debt*, 2026, arXiv:2601.07786.
7. K. Borowa, A. Zalewski, and S. Kijas, "The influence of cognitive biases on architectural technical debt," in *Proc. IEEE 18th Int. Conf. Softw. Archit. (ICSA)*, 2021, doi: 10.1109/ICSA51549.2021.00019.
8. X. Zhou, Z. Saghi, S. Sabouri, R. Pandita, M. McGuire, and S. Chattopadhyay, "Cognitive biases in LLM-assisted software development," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, doi: 10.1145/3744916.3773104.
9. T. O'Donoghue and M. Rabin, "Doing it now or later," *Amer. Econ. Rev.*, vol. 89, no. 1, pp. 103–124, 1999.
10. R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010.
11. D. Kahneman and A. Tversky, "Prospect theory: An analysis of decision under risk," *Econometrica*, vol. 47, no. 2, pp. 263–291, 1979.
12. A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *Proc. IEEE Int. Conf. Softw. Maint. Evol. (ICSME)*, 2014, pp. 91–100.
13. H. Li, H. Zhang, and A. E. Hassan, "AIDev: Studying AI coding agents on GitHub," in *Proc. 23rd Int. Conf. Mining Softw. Repositories*, 2026, doi: 10.1145/3793302.3797249.

OOK LEE is a [position] in the [department] at Hanyang University, Seoul, 04763, South Korea. His research interests include [interest 1], [interest 2],

and [interest 3]. Lee received the [degree] in [field] from [institution]. He is a member of [IEEE / IEEE Computer Society]. Contact him at ooklee@hanyang.ac.kr.